

DJAMA ERP

# AUDIT COMPLET

## MODULE CHRONO PRO

---

Rapport généré le 30 September 2026

Confidentiel — Usage interne

Ce document présente l'audit exhaustif du module Chrono Pro de l'ERP DJAMA. Il couvre l'architecture existante, la sécurité (RLS, API, multi-tenant), les fonctionnalités, les calculs temps/facturation, les intégrations inter-modules, l'UX/design et la roadmap de mise en conformité.

# 1. Résumé Exécutif

| Dimension                      | État                                                   | Priorité |
|--------------------------------|--------------------------------------------------------|----------|
| Architecture (routes API)      | ■ Aucune route API — tout passe par le client Supabase | P0       |
| Multi-tenant / organisation_id | ■ Absent sur toutes les tables Chrono                  | P0       |
| Sécurité calcul durée          | ■ Calculée côté navigateur — falsifiable               | P0       |
| Sécurité taux horaire          | ■ Aucune validation serveur — falsifiable              | P0       |
| Sécurité création facture      | ■ Montants calculés côté client, insertion directe     | P0       |
| RLS (politiques d'accès)       | ■ Actives — mais doublon de policies dans 2 migrations | P1       |
| Persistence timer (refresh)    | ■ Timer perdu sur navigation/refresh                   | P1       |
| Édition d'une entrée           | ■ Absente                                              | P1       |
| Connexion CRM / Projets ERP    | ■ Champs texte libre sans FK                           | P2       |
| Connexion Productivité         | ■ Absente                                              | P2       |
| Connexion Factures & Devis     | ■ Partielle — insertion sans validation serveur        | P0       |
| Équipe / multi-utilisateurs    | ■ Absent                                               | P2       |
| UX / Design                    | ■ Bonne base — violet prédominant vs Gold DJAMA        | P3       |
| Modes (Chrono / Pomodoro...)   | ■ 4 modes fonctionnels                                 | —        |
| Statistiques                   | ■ Données réelles — calculs frontend corrects          | —        |
| Export ICS                     | ■ Fonctionnel côté client                              | —        |
| Objectifs                      | ■ Fonctionnels (quotidien / hebdo / facturable)        | —        |

**Verdict :** Le module Chrono Pro est fonctionnellement riche avec une UX soignée, mais repose sur une architecture sans couche serveur. Les calculs financiers (durée, taux horaire, montant facturé) sont intégralement contrôlés par le navigateur. Il ne répond pas aux exigences d'un ERP multi-tenant sécurisé. Avant toute évolution fonctionnelle, les problèmes **P0** (sécurité et architecture) doivent être résolus.

## 2. Architecture Existante

### 2.1 Fichiers du module

| Rôle                       | Fichier                                           |
|----------------------------|---------------------------------------------------|
| Page principale (frontend) | src/app/client/chrono/page.tsx                    |
| Routes API dédiées         | ■ Aucune — 0 fichier dans src/app/api/chrono/     |
| Migration tables Chrono    | supabase/migrations/chrono_schema.sql             |
| Migration tables générales | supabase/migrations/018_new_tools.sql             |
| Migration tables générales | supabase/migrations/021_create_missing_tables.sql |
| Migration RLS globale      | supabase/migrations/20260701_rls_policies.sql     |

### 2.2 Flux de données

Le module Chrono n'utilise **aucune route API Next.js**. Toutes les opérations (lecture, écriture, mise à jour, suppression) passent par le SDK Supabase JavaScript côté navigateur, avec la clé *anon* et la protection RLS. Il n'existe donc aucune couche de validation métier entre le navigateur et la base de données.

| Composant                | Mécanisme                      | Risque                                  |
|--------------------------|--------------------------------|-----------------------------------------|
| Lecture time_entries     | supabase.from(...).select()    | Faible — RLS filtre par user_id         |
| Insertion entrée         | supabase.from(...).insert()    | <b>CRITIQUE</b> — durée calculée client |
| Suppression entrée       | supabase.from(...).delete()    | Faible — RLS empêche cross-user         |
| Création projet Chrono   | supabase.from(...).insert()    | Faible                                  |
| Création facture (temps) | supabase.from(...).insert() ×3 | <b>CRITIQUE</b> — montants client-side  |
| Marquage 'is_billed'     | supabase.from(...).update()    | <b>Moyen</b> — race condition possible  |
| Objectifs                | supabase.from(...).upsert()    | Faible                                  |

## 3. Base de Données & RLS

### 3.1 Tables Chrono — tableau de sécurité

| TABLE           | organization_id | user_id     | RLS | POLICIES                            | RISQUE |
|-----------------|-----------------|-------------|-----|-------------------------------------|--------|
| time_entries    | ■               | ■           | ■   | 5 policies (doublon × 2 migrations) | MOYEN  |
| chrono_projects | ■               | ■           | ■   | 5 policies (doublon × 2 migrations) | MOYEN  |
| chrono_goals    | ■               | ■<br>UNIQUE | ■   | 5 policies (doublon × 2 migrations) | FAIBLE |

### 3.2 Analyse détaillée — table time\_entries

| Colonne          | Type         | Nullable | Rôle / Observation                                |
|------------------|--------------|----------|---------------------------------------------------|
| id               | uuid PK      | Non      | Clé primaire auto-générée                         |
| user_id          | uuid FK      | Non      | FK auth.users — base de la RLS                    |
| project          | text         | Non      | ■ Texte libre — pas de FK vers chrono_projects    |
| client_name      | text         | Oui      | ■ Texte libre — pas de FK vers contacts CRM       |
| description      | text         | Oui      | Description libre de la session                   |
| task_title       | text         | Oui      | ■ Texte libre — pas de FK vers productivity_tasks |
| date             | date         | Non      | Date de la session (pas de timezone)              |
| duration_minutes | integer      | Non      | ■ Calculé côté client — aucune validation max     |
| hourly_rate      | numeric 10,2 | Oui      | ■ Saisi côté client — aucun plafond serveur       |
| category         | text         | Oui      | Catégorie libre                                   |
| is_billable      | boolean      | Non      | Flag facturable                                   |
| is_billed        | boolean      | Non      | Flag déjà facturé                                 |
| invoice_ref      | text         | Oui      | Référence facture (text libre)                    |
| timer_mode       | text         | Oui      | classique / pomodoro / countdown / focus          |
| notes            | text         | Oui      | Notes libres                                      |
| created_at       | timestampz   | Non      | Timestamp création                                |
| updated_at       | timestampz   | Non      | Timestamp MAJ                                     |

### 3.3 Problème d'isolation multi-tenant

Aucune des tables Chrono (*time\_entries*, *chrono\_projects*, *chrono\_goals*) ne possède de colonne **organization\_id**. Les RLS sont actuellement basées uniquement sur `user_id = auth.uid()`, ce qui isole les utilisateurs entre eux mais **ne permet pas l'isolation au niveau de l'organisation**. Dans un contexte ERP multi-tenant où plusieurs employés appartiennent à la même organisation (partagent un workspace), il est impossible aujourd'hui de :

- permettre à un manager de voir le temps de son équipe,
- différencier les permissions entre membres de la même organisation,

- cloisonner les données de deux organisations distinctes sur la même instance.

## 4. Sécurité & API

### 4.1 Problèmes critiques — P0

#### ■ AUCUNE ROUTE API SERVEUR

Toutes les mutations passent par le SDK Supabase anon côté navigateur. Il n'y a aucune couche intermédiaire pour valider les données métier. Un attaquant avec les DevTools peut appeler directement l'API Supabase.

#### ■ DURÉE MANIPULABLE CÔTÉ CLIENT

La durée (`duration_minutes`) est calculée en JavaScript via `Math.floor((Date.now() - startMs) / 1000)`. Tout utilisateur peut insérer manuellement `duration_minutes=99999` via la console ou un script, sans aucune validation.

#### ■ TAUX HORAIRE NON VALIDÉ CÔTÉ SERVEUR

Le champ `hourly_rate` est un sans plafond côté serveur. Un utilisateur peut enregistrer `hourly_rate=999999` et voir des revenus fictifs.

#### ■ CRÉATION DE FACTURE SANS VALIDATION SERVEUR

`handleCreateInvoice()` calcule `totalHt` et `tva20` côté client, puis insère directement dans la table `documents` sans aucune route API. Les montants financiers ne sont jamais recalculés côté serveur. Une facture peut contenir des montants falsifiés.

#### ■ ABSENCE D'ORGANIZATION\_ID

Impossible de mettre en place des permissions équipe, un audit par organisation, ou un cloisonnement multi-tenant. Tout ajout de fonctionnalités collaboratives nécessite une migration majeure de schéma.

### 4.2 Problèmes moyens — P1

#### ■ TIMER NON PERSISTÉ SUR NAVIGATION/REFRESH

Le timer en cours est stocké uniquement dans l'état React (`useState`). Un refresh ou une navigation vers un autre module détruit le timer sans avertissement. Il faut persister `started_at` et `elapsed` en `localStorage` ou en base.

#### ■ RACE CONDITION — DOUBLE FACTURATION

`handleCreateInvoice()` n'utilise pas de transaction atomique. Si déclenché simultanément depuis deux onglets, deux factures peuvent être créées pour les mêmes `time_entries`. La protection `creatingInv` est uniquement en mémoire React.

#### ■ AUCUN RATE LIMITING

Sans route API, il est impossible d'ajouter un rate limiting standard. Un utilisateur peut spammer des insertions (milliers d'entrées) sans limite.

#### ■ DOUBLON DE POLICIES RLS

time\_entries a deux jeux de policies : time\_entries\_own (migration 018) et user\_select/insert/update/delete\_time\_entries (migration 20260701). Même résultat fonctionnel, mais gouvernance confuse. À nettoyer.

#### ■ TVA HARDCODÉE À 20%

La création de facture applique systématiquement 20% de TVA sans vérification du profil fiscal (auto-entrepreneur exonéré, TVA sur marge, etc.).

#### ■ CHAMPS ÉMETTEUR VIDES À LA CRÉATION DE FACTURE

La facture créée depuis Chrono a emetteur\_nom, emetteur\_email, etc. à vide. L'utilisateur doit les compléter manuellement dans le module Factures.

#### ■ ÉDITION D'ENTRÉE ABSENTE

Impossible de modifier une time\_entry existante. L'utilisateur doit supprimer et recréer, ce qui peut entraîner des pertes de données ou des doublons.

# 5. Audit des Fonctionnalités

## 5.1 Chronomètre

| Fonctionnalité               | État      | Observation                                  |
|------------------------------|-----------|----------------------------------------------|
| Démarrer                     | ■ COMPLET | Fonctionnel                                  |
| Pause                        | ■ COMPLET | Fonctionnel                                  |
| Reprendre                    | ■ COMPLET | Recalcule correctement le startMs            |
| Arrêter + sauvegarder        | ■ COMPLET | saveSession() appelée automatiquement        |
| Annuler                      | ■ COMPLET | Remet le timer à zéro sans sauvegarder       |
| Persistance après navigation | ■ ABSENT  | Timer perdu — stocké uniquement en useState  |
| Persistance après refresh    | ■ ABSENT  | Même problème — aucun localStorage ni DB     |
| Protection multi-onglets     | ■ ABSENT  | Aucun verrou en base ou localStorage         |
| Calcul durée côté serveur    | ■ ABSENT  | Date.now() navigateur uniquement             |
| Ajout manuel d'une entrée    | ■ COMPLET | Modale complète avec tous les champs         |
| Modification d'une entrée    | ■ ABSENT  | Supprimer + recréer uniquement               |
| Suppression                  | ■ COMPLET | Confirmation demandée                        |
| Historique                   | ■ COMPLET | 500 entrées groupées par date                |
| Gestion fuseau horaire       | ■ PARTIEL | date stockée sans timezone — risque décalage |

## 5.2 Modes

| Mode             | État      | Fonctionnement réel                                                                                       |
|------------------|-----------|-----------------------------------------------------------------------------------------------------------|
| Classique        | ■ COMPLET | Chronomètre ascendant libre — fonctionne correctement                                                     |
| Pomodoro         | ■ COMPLET | 25 min travail / 5 min pause, cycles comptés, sauvegarde auto à chaque pomodoro terminé, notification nav |
| Compte à rebours | ■ COMPLET | Durée configurable (h + min), arrêt automatique et sauvegarde                                             |
| Focus            | ■ COMPLET | Overlay plein écran immersif, affichage revenus estimés en temps réel                                     |

## 5.3 Temps facturable

| Élément                         | État      | Observation                             |
|---------------------------------|-----------|-----------------------------------------|
| Toggle facturable               | ■ COMPLET | Visible sur timer + ajout manuel        |
| Taux horaire (saisie)           | ■ COMPLET | ■ Sans validation serveur — falsifiable |
| Calcul montant (client-side)    | ■ COMPLET | duration_minutes / 60 × hourly_rate     |
| Validation montant côté serveur | ■ ABSENT  | Aucun recalcul serveur — CRITIQUE       |
| Badge 'FACTURÉ' dans historique | ■ COMPLET | Visible sur les entrées marquées        |
| Filtre entrées non facturées    | ■ COMPLET | Onglet Facturable fonctionnel           |

|                                 |           |                                      |
|---------------------------------|-----------|--------------------------------------|
| Regroupement par client         | ■ COMPLET | Dans l'onglet Facturable             |
| Arrondis                        | ■ COMPLET | Math.round à 2 décimales             |
| Sessions déjà facturées exclues | ■ COMPLET | Filtre is_billed=false dans l'onglet |
| Validation taux max             | ■ ABSENT  | Pas de plafond serveur               |

## 6. Calculs Temps & Facturation

### 6.1 Flux de création de facture depuis Chrono

La fonction `handleCreateInvoice()` effectue les opérations suivantes, toutes côté client sans aucune route API :

1. Calcule  $\text{totalHt} = \text{somme}(\text{duration\_minutes} / 60 \times \text{hourly\_rate})$  — côté client
2. Calcule  $\text{tva20} = \text{totalHt} \times 0.20$  — TVA hardcodée à 20%
3. Calcule  $\text{totalTtc} = \text{totalHt} + \text{tva20}$  — côté client
4. Insère une ligne dans `documents` (`type='facture'`) avec ces montants
5. Insère des lignes dans `document_items` — une par `time_entry`
6. Met à jour `time_entries` SET `is_billed=true`, `invoice_ref=`
7. Redirige vers `/client/factures`

■ **Risque critique** : Étapes 1, 2, 3, 4, 5 se déroulent sans recalcul serveur. Un utilisateur malveillant ou une erreur de synchronisation peut entraîner une facture avec des montants incorrects enregistrés en base. La solution correcte est une route API POST `/api/chrono/create-invoice` qui récupère les `time_entries` depuis la base, recalcule les montants et crée la facture atomiquement.

### 6.2 Protection contre la double facturation

| Mécanisme                                            | Type            | Efficacité                                |
|------------------------------------------------------|-----------------|-------------------------------------------|
| <code>creatingInv</code> (state React)               | Frontend only   | Protège contre le double-clic simple      |
| <code>is_billed=true</code> après création           | Base de données | Marque les entrées comme facturées        |
| Filtre <code>is_billed=false</code> dans l'UI        | Frontend only   | Cache les entrées déjà facturées          |
| Transaction atomique ( <code>upsert/rpc</code> )     | ■ Absent        | ■ Race condition entre 2 onglets possible |
| Vérif. préalable <code>is_billed</code> avant INSERT | ■ Absent        | Pas de guard côté serveur                 |

# 7. Intégrations Inter-Modules ERP

| Module           | État      | Détail                                                                                                       |
|------------------|-----------|--------------------------------------------------------------------------------------------------------------|
| CRM / Clients    | ■ ABSENT  | client_name = champ texte libre. Pas de FK vers contacts. Pas d'autocomplétion.                              |
| Projets ERP      | ■ PARTIEL | Bouton 'Chrono' dans /projets navigue vers /chrono sans passer de données. Chrono utilise sa propre table ch |
| Productivité     | ■ ABSENT  | Aucun lien vers productivity_tasks. task_title = texte libre.                                                |
| Factures & Devis | ■ PARTIEL | Chrono peut créer une facture dans documents, mais sans validation serveur ni données émetteur préremplies   |
| Équipe           | ■ ABSENT  | Aucune fonctionnalité multi-membres. Pas de FK vers team_members.                                            |
| Planning         | ■ ABSENT  | Export ICS côté client uniquement. Pas de lecture/écriture dans planning_events.                             |
| Notifications    | ■ ABSENT  | Pas d'intégration push_subscriptions. Notifications navigateur uniquement (Pomodoro).                        |
| Comptabilité     | ■ ABSENT  | Aucun lien.                                                                                                  |

Architecture cible recommandée : Client DJAMA → Projet DJAMA → Tâche Productivité → Session Chrono (time\_entry). Les FK doivent pointer vers les tables existantes (contacts, projects, productivity\_tasks) avec texte de fallback si non lié.

## 8. Audit UX & Design

### 8.1 État actuel

| Élément                      | État       | Observation                                                                         |
|------------------------------|------------|-------------------------------------------------------------------------------------|
| Cohérence fond dark #07080e  | ■ COMPLET  | Conforme au design system DJAMA                                                     |
| Couleur dominante DJAMA GOLD | ■ PARTIEL  | L'accent violet #a78bfa (Tailwind violet-400) domine. Gold présent mais secondaire. |
| Timer visuel animé           | ■ COMPLET  | Glow animé + changement de couleur selon mode/état                                  |
| Revenus en temps réel        | ■ COMPLET  | Estimation € au fil des secondes en mode Focus                                      |
| Mode Focus (overlay)         | ■ COMPLET  | Plein écran immersif bien exécuté                                                   |
| Animations Framer Motion     | ■ COMPLET  | Spring physics sur modales — fluides                                                |
| Dark / Light mode            | ■ COMPLET  | Support complet via ThemeContext                                                    |
| Responsive mobile            | ■ PARTIEL  | Adaptatif mais non spécifiquement optimisé                                          |
| Densité informations         | ■ PARTIEL  | KPI trop grandes sur desktop — espace inutile                                       |
| Édition in-line (historique) | ■ ABSENT   | Supprimer + recréer uniquement                                                      |
| Skeleton / loading states    | ■ PARTIEL  | Spinner basique — pas de skeleton screen                                            |
| États vides (empty states)   | ■ COMPLET  | Illustrés avec icônes et call-to-action                                             |
| Confirmations destructives   | ■ COMPLET  | Modale de confirmation avant suppression                                            |
| Étiquette 'Analyse IA'       | ■ TROMPEUR | Labellisé IA avec icône Brain — ce sont des règles if/else déterministes            |
| Connexion visuelle modules   | ■ ABSENT   | Aucun lien visuel vers CRM, Projets, Productivité                                   |

### 8.2 Recommandations design

- Remplacer l'accent violet (#a78bfa) par Gold DJAMA (#c9a55a) comme couleur principale du timer.
- Réduire la taille des KPI cards — trop de hauteur, trop peu de densité.
- Ajouter un skeleton screen (pulsation) au lieu du spinner lors du chargement initial.
- Renommer 'Analyse IA Productivité' en 'Analyse statistique' ou 'Insights' pour être honnête.
- Ajouter une barre de statut globale visible qui indique si un timer tourne (toutes pages).
- Ajouter l'édition in-line des entrées dans l'historique (crayon).
- Sur mobile : réorganiser le timer + formulaire projet/client en scroll unique, sans onglets.
- Pré-remplir les champs émetteur de la facture depuis le profil DJAMA.

## 9. Classification des Issues

| P0 Sécurité & Données | Élément                                   | Action requise                                                             |
|-----------------------|-------------------------------------------|----------------------------------------------------------------------------|
|                       | Aucune route API serveur pour Chrono      | Créer des routes API Next.js pour toutes les mutations                     |
|                       | Durée calculée côté client                | Calculer duration côté serveur à partir de started_at/ended_at             |
|                       | Taux horaire non validé serveur           | Valider et plafonner hourly_rate côté serveur                              |
|                       | Création facture sans validation serveur  | Route POST /api/chrono/create-invoice avec recalcul complet                |
|                       | Absence d'organization_id                 | Migrer les tables pour ajouter organization_id (migration non-destructive) |
|                       | Race condition double facturation         | Transaction atomique avec vérification préalable                           |
| P1 Fonctionnelles     | Élément                                   | Action requise                                                             |
|                       | Timer non persisté sur refresh/navigation | Persister started_at et elapsed en localStorage + DB                       |
|                       | Édition d'une entrée impossible           | Ajouter une modale d'édition (PUT /api/chrono/entry/:id)                   |
|                       | Doublon de polices RLS                    | Supprimer les anciennes polices dans 018_new_tools.sql                     |
|                       | TVA hardcodée à 20%                       | Lire le profil fiscal de l'organisation                                    |
|                       | Champs émetteur vides dans la facture     | Pré-remplir depuis le profil organisation                                  |
|                       | Validation timezone des dates             | Stocker timestamp complet ou timezone explicite                            |
| P2 Intégrations       | Élément                                   | Action requise                                                             |
|                       | CRM : client_name texte libre             | Ajouter FK optionnel vers contacts + autocomplétion                        |
|                       | Projets ERP non connectés                 | Ajouter FK optionnel vers projects + synchroniser chrono_projects          |
|                       | Productivité non connectée                | Ajouter FK optionnel vers productivity_tasks                               |
|                       | Équipe : zéro multi-membres               | Permissions explicites basées sur organization_id                          |
|                       | Planning : export ICS uniquement          | Lecture/écriture optionnelle dans planning_events                          |
|                       | Notifications : navigateur uniquement     | Intégrer push_subscriptions pour fin de Pomodoro                           |

| P3 Design & Finitions | Élément                       | Action requise                                               |
|-----------------------|-------------------------------|--------------------------------------------------------------|
|                       | Violet dominant vs Gold DJAMA | Remplacer accent violet par Gold sur les éléments principaux |
|                       | Densité KPI trop basse        | Réduire hauteur des cartes, augmenter densité                |
|                       | Skeleton loading absent       | Ajouter skeleton screen au chargement initial                |
|                       | Label 'Analyse IA' trompeur   | Renommer en 'Insights statistiques'                          |
|                       | Barre timer globale absente   | Indicateur persistant si timer actif (layout global)         |
|                       | Responsive mobile perfectible | Refonte layout mobile — scroll unique sans onglets           |

## 10. Roadmap Complète

|         |                                                                                                      |
|---------|------------------------------------------------------------------------------------------------------|
| Phase 1 | <b>DB + Multi-tenant + RLS</b>                                                                       |
|         | • Ajouter organization_id (NOT NULL, FK organizations) à time_entries, chrono_projects, chrono_goals |
|         | • Migration non-destructive : valeur par défaut = organisation courante de l'utilisateur             |
|         | • Mettre à jour toutes les politiques RLS : organization_id = get_user_organization_id()             |
|         | • Supprimer les politiques en doublon des migrations 018 et 20260701                                 |
|         | • Ajouter indexes sur (organization_id, user_id) pour les performances                               |
|         | • Ajouter colonne started_at (timestampz) et ended_at (timestampz) à time_entries                    |
| Phase 2 | • Tests : vérifier isolation stricte entre organisations                                             |
|         | <b>Moteur Chrono fiable + Sessions</b>                                                               |
|         | • Créer route POST /api/chrono/start — génère un timer_session avec started_at côté serveur          |
|         | • Créer route POST /api/chrono/stop — calcule duration = ended_at - started_at côté serveur          |
|         | • Créer route POST /api/chrono/pause + /resume avec gestion des pauses en DB                         |
|         | • Persister l'état du timer en cours en localStorage (fallback) + timer_sessions en DB               |
|         | • Afficher warning si session non terminée détectée au rechargement                                  |
| Phase 3 | • Protection contre sessions simultanées : 1 session active par utilisateur                          |
|         | • Créer route PUT /api/chrono/entry/:id pour l'édition                                               |
|         | • Créer route DELETE /api/chrono/entry/:id                                                           |
|         | <b>Clients + Projets + Tâches</b>                                                                    |
|         | • Ajouter FK optionnel contact_id (vers contacts) à time_entries avec texte fallback                 |
|         | • Ajouter FK optionnel project_id (vers projects) à time_entries et chrono_projects                  |
|         | • Ajouter FK optionnel task_id (vers productivity_tasks) à time_entries                              |
|         | • Autocomplétion client depuis contacts CRM (debounced, organisation filtrée)                        |
|         | • Autocomplétion projet depuis projects ERP + chrono_projects                                        |
|         | • Autocomplétion tâche depuis productivity_tasks filtrées par projet                                 |
|         | • Synchro bidirectionnelle : un projet ERP créé apparaît dans Chrono                                 |

|         |                                                                                                                                                                                                                       |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Phase 4 | <b>Facturable + Taux + Calculs serveur</b>                                                                                                                                                                            |
|         | <ul style="list-style-type: none"> <li>• Valider hourly_rate côté serveur (route API, plafond configurable par organisation)</li> </ul>                                                                               |
|         | <ul style="list-style-type: none"> <li>• Stocker currency sur time_entries (défaut EUR)</li> </ul>                                                                                                                    |
|         | <ul style="list-style-type: none"> <li>• Lire le taux horaire depuis le projet ou le profil organisation si non saisi</li> </ul>                                                                                      |
|         | <ul style="list-style-type: none"> <li>• Recalculer systématiquement côté serveur : <math>\text{montant\_ht} = \text{ROUND}(\text{duration\_minutes}::\text{numeric} / 60 * \text{hourly\_rate}, 2)</math></li> </ul> |
|         | <ul style="list-style-type: none"> <li>• Ajouter arrondi configurable (par 6 min, par 15 min, exact)</li> </ul>                                                                                                       |
| Phase 5 | <b>Factures &amp; Devis</b>                                                                                                                                                                                           |
|         | <ul style="list-style-type: none"> <li>• Créer route POST /api/chrono/create-invoice avec recalcul complet côté serveur</li> </ul>                                                                                    |
|         | <ul style="list-style-type: none"> <li>• Utiliser une transaction atomique : vérifier is_billed=false BEFORE INSERT</li> </ul>                                                                                        |
|         | <ul style="list-style-type: none"> <li>• Pré-remplir emetteur_* depuis le profil organisation</li> </ul>                                                                                                              |
|         | <ul style="list-style-type: none"> <li>• Appliquer le taux TVA depuis le profil fiscal de l'organisation (pas hardcodé à 20%)</li> </ul>                                                                              |
|         | <ul style="list-style-type: none"> <li>• Marquer time_entries IS BILLED dans la même transaction</li> </ul>                                                                                                           |
|         | <ul style="list-style-type: none"> <li>• Ajouter lien retour : dans le module Factures, voir les time_entries d'une facture</li> </ul>                                                                                |
|         | <ul style="list-style-type: none"> <li>• Bloquer côté API la re-facturation d'entrées déjà is_billed=true</li> </ul>                                                                                                  |
| Phase 6 | <b>Stats + Objectifs + Rapports</b>                                                                                                                                                                                   |
|         | <ul style="list-style-type: none"> <li>• Déplacer les calculs stats complexes vers des RPCs Supabase (performance)</li> </ul>                                                                                         |
|         | <ul style="list-style-type: none"> <li>• Ajouter objectif mensuel et objectif de CA dans chrono_goals</li> </ul>                                                                                                      |
|         | <ul style="list-style-type: none"> <li>• Stats par membre pour les managers (basé sur organization_id + permissions)</li> </ul>                                                                                       |
|         | <ul style="list-style-type: none"> <li>• Export PDF du rapport (route API + puppeteer ou reportlab Python)</li> </ul>                                                                                                 |
|         | <ul style="list-style-type: none"> <li>• Export XLSX (feuille par client, feuille récap)</li> </ul>                                                                                                                   |
|         | <ul style="list-style-type: none"> <li>• Filtres rapport : période + client + projet + membre + facturable</li> </ul>                                                                                                 |
| Phase 7 | <b>Équipe + Planning + Permissions</b>                                                                                                                                                                                |
|         | <ul style="list-style-type: none"> <li>• Ajouter table chrono_permissions (user_id, target_user_id, can_read_time, can_read_billing)</li> </ul>                                                                       |
|         | <ul style="list-style-type: none"> <li>• Vue manager : voir temps de l'équipe selon permissions explicites</li> </ul>                                                                                                 |
|         | <ul style="list-style-type: none"> <li>• Un membre ne voit que son propre temps par défaut</li> </ul>                                                                                                                 |
|         | <ul style="list-style-type: none"> <li>• Intégration optionnelle planning_events : créer un event sur démarrage/arrêt</li> </ul>                                                                                      |
|         | <ul style="list-style-type: none"> <li>• Notifications push (fin de Pomodoro, objectif atteint) via push_subscriptions</li> </ul>                                                                                     |

| Phase 8 | UX/Design Premium                                                                              |
|---------|------------------------------------------------------------------------------------------------|
|         | • Remplacer l'accent violet par Gold DJAMA sur le timer et les éléments principaux             |
|         | • Refonte layout : timer + formulaire en colonne, historique en full-width dessous             |
|         | • Skeleton screens au chargement initial                                                       |
|         | • Éditeur in-line dans l'historique (clic sur une entrée pour éditer)                          |
|         | • Barre de timer persistante dans le layout global (visible sur toutes les pages)              |
|         | • Renommer 'Analyse IA' en 'Insights'                                                          |
|         | • Optimisation mobile : scroll unique, formulaire condensé                                     |
|         | • Dark/Light : vérifier les contrastes sur tous les nouveaux composants                        |
| Phase 9 | Tests + Sécurité + Audit final                                                                 |
|         | • Tests d'isolation : Organisation A ne voit pas les données de B                              |
|         | • Tests de permissions : employé ne voit pas le temps d'un autre membre sans autorisation      |
|         | • Tests de manipulation : tenter d'insérer duration=99999 ou hourly_rate=99999 via API directe |
|         | • Tests de race condition : double-clic simultané sur 'Créer facture'                          |
|         | • Audit RLS exhaustif sur toutes les nouvelles tables                                          |
|         | • Test de performance : 10 000 time_entries par organisation                                   |
|         | • Test export ICS : compatibilité Google Calendar, Outlook, Apple Calendar                     |
|         | • Revue de code sécurité des routes API créées                                                 |

## 11. Risques de Migration

### ■ CRITIQUE — Ajout organization\_id aux tables existantes

Les time\_entries existantes n'ont pas de organization\_id. La migration doit déduire l'organisation depuis user\_id via organization\_members. Si un utilisateur appartient à plusieurs organisations, une règle de priorité est nécessaire. Migration idempotente obligatoire. NE PAS déclarer la colonne NOT NULL avant le backfill.

### ■ CRITIQUE — Remplacement calcul durée client → serveur

Les time\_entries existantes ont uniquement duration\_minutes (calculé côté client). Les nouvelles colonnes started\_at / ended\_at n'existent pas encore. Migration : ajouter les colonnes nullable, puis peupler rétrospectivement si possible (approximation : started\_at = created\_at, ended\_at = created\_at + duration). Ne pas supprimer duration\_minutes — colonne de fallback.

### ■ MOYEN — Suppression des politiques RLS en doublon

time\_entries a deux jeux de politiques. Supprimer l'ancienne (time\_entries\_own) après avoir vérifié que les nouvelles (user\_select/insert/update/delete) couvrent tous les cas. Tester en staging avant production.

### ■ MOYEN — Migration chrono\_projects vers projects ERP

Les projets Chrono sont indépendants des projets ERP. Avant de remplacer par des FK, un mapping manuel ou semi-automatique (par nom) est nécessaire. Les noms peuvent différer. Garder le texte en fallback.

### ■ FAIBLE — Routes API — rupture de compatibilité frontend

Le passage de mutations directes Supabase à des routes API nécessite de mettre à jour tous les appels supabase.from('time\_entries').insert() dans page.tsx. Refactor page par page, avec tests manuels.

## 12. Tests Nécessaires

### Sécurité & isolation

- Insérer directement une time\_entry avec duration\_minutes=99999 via Supabase SDK — vérifier rejet
- Insérer directement avec hourly\_rate=999999 — vérifier rejet
- Lire les time\_entries d'un autre user\_id — vérifier 0 résultats (RLS)
- Lire les time\_entries d'une autre organisation — vérifier 0 résultats
- Créer une facture avec montants falsifiés depuis l'API — vérifier recalcul serveur
- Tenter de marquer is\_billed=true d'une entrée d'un autre utilisateur — vérifier rejet

### Timer & sessions

- Démarrer un timer, naviguer vers un autre module, revenir — vérifier persistance
- Démarrer un timer, rafraîchir la page — vérifier que la session reprend ou avertit
- Ouvrir deux onglets, démarrer un timer dans chacun — vérifier blocage du second
- Laisser tourner un timer 60 min — vérifier durée exacte vs started\_at DB
- Démarrer en mode Pomodoro — vérifier cycles, pauses, sauvegarde auto
- Démarrer un compte à rebours, le laisser expirer — vérifier sauvegarde auto

### Facturation

- Sélectionner des entrées et créer une facture — vérifier que is\_billed=true
- Tenter de facturer à nouveau les mêmes entrées — vérifier blocage
- Double-clic rapide sur 'Créer facture' — vérifier une seule facture créée
- Vérifier que la facture créée a les montants corrects recalculés côté serveur
- Vérifier que les champs émetteur sont préremplis depuis le profil

### Intégrations

- Sélectionner un client CRM existant dans le timer — vérifier l'autocomplétion
- Sélectionner un projet ERP existant — vérifier la liaison FK
- Lancer un chrono depuis la page Productivité (tâche) — vérifier le passage du contexte
- Exporter ICS — importer dans Google Calendar et vérifier dates + timezone

### Performance

- Charger la page avec 10 000 time\_entries — vérifier temps de chargement < 3s
- Vérifier indexes sur (organization\_id, user\_id, date) sur time\_entries
- Vérifier que les stats ne recalculent pas toutes les entrées à chaque rendu

## 13. Recommandations

Le module Chrono Pro est fonctionnellement riche et visuellement soigné. Sa refonte doit suivre un ordre strict : **sécurité et architecture d'abord, intégrations ensuite, design en dernier**. Voici les 10 recommandations prioritaires :

|    |                                                                                                                                                                                                                |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | <b>[P0] Créer une couche API serveur complète</b><br>Toutes les mutations Chrono doivent passer par des routes Next.js /api/chrono/. C'est le prérequis de tous les autres points de sécurité.                 |
| 2  | <b>[P0] Calculer la durée côté serveur</b><br>Stocker started_at/ended_at en DB. La durée est calculée par la route API stop, jamais envoyée par le client.                                                    |
| 3  | <b>[P0] Recalculer les montants facturés côté serveur</b><br>La route /api/chrono/create-invoice doit relire les time_entries depuis la DB, recalculer totalHt et insérer atomiquement.                        |
| 4  | <b>[P0] Ajouter organization_id aux tables Chrono</b><br>Migration non-destructive avec backfill depuis organization_members. Prérequis pour toute fonctionnalité équipe.                                      |
| 5  | <b>[P1] Persister le timer en cours</b><br>localStorage pour l'UI immédiate + une table timer_sessions en DB pour la fiabilité.                                                                                |
| 6  | <b>[P1] Ajouter l'édition d'une entrée</b><br>Route PUT /api/chrono/entry/:id avec validation complète. Modale d'édition dans l'historique.                                                                    |
| 7  | <b>[P2] Connecter CRM + Projets ERP + Productivité</b><br>FK optionnelles avec fallback texte. Autocomplétion depuis les données DJAMA existantes. Ne pas dupliquer les données.                               |
| 8  | <b>[P2] Sécuriser la facturation contre la double facturation</b><br>Transaction atomique avec vérification préalable is_billed=false. Lecture du profil fiscal pour la TVA.                                   |
| 9  | <b>[P3] Aligner l'identité visuelle sur DJAMA Gold</b><br>L'accent violet doit céder la place au Gold #c9a55a comme couleur principale du timer. Chrono doit rester un module DJAMA, pas une app indépendante. |
| 10 | <b>[P3] Renommer 'Analyse IA' en 'Insights'</b><br>La section utilise des règles déterministes if/else. Labelliser comme 'IA' est trompeur.                                                                    |